When Apparent Updates Still Depend on History

Anonymous ACL submission

Abstract

Evaluating a model's reasoning after an update requires that the update carries through to downstream answers. However, existing approaches still encounter downstream failures. We find model answers can depend on outdated facts, even after those facts are updated. We investigate this using *histories* – the starting sets of facts given to a model. We construct *matched histories* by applying the same in-context updates or activation interventions to two histories, so both end with the same facts, and ask both the same downstream question. We restrict to cases where the model reports the updated facts correctly and answers the question given those facts from the start. Even under strict controls and on public MQuAKE multi-hop questions, history dependence remains, affecting up to 45% of cases. We attempted to causally isolate it, but found effects spread across multiple transformer blocks, making removal difficult without disrupting correct behavior. Instead, we propose a neurosymbolic repair that stores updated facts in an external dictionary, with a trained interpreter mapping each question to a dictionary operation. This eliminates history dependence while answering, on average, 99% of MQuAKE cases correctly, versus 38% when given only corrected facts.

1 Introduction

Language models are routinely updated after training. These updates take a variety of forms, ranging from knowledge editing (Meng et al., 2022, 2023) to corrections given in context, and even changing state in agents and memory systems (Packer et al., 2023). Beyond just accuracy, one especially relevant application is in AI safety, where an update is meant to remove a harmful belief. To check if model updates have successfully propagated, the updated facts are checked to ensure they are reported correctly. While the check is able to demonstrate that the new fact is accessible and producible, it does not guarantee that the model successfully uses this fact in its later answers. If the fact replaces a harmful belief, the model would pass this check and could still act on that belief. In fact, knowledge-editing benchmarks have already documented failures on multi-hop and related questions after apparently successful edits (Zhong et al., 2023; Cohen et al., 2024; Ju et al., 2024). However, these benchmarks do not distinguish the origin of the failures, which only invites uncertainty in the effectiveness of newly proposed solutions. We address this gap by isolating dependence on outdated information as a distinct source of downstream failure. This leads us to ask: if distinct histories – the starting sets of facts given to a model – receive identical updates and reach the same final facts, with the facts required by a downstream question reported correctly under every history, will the downstream answers still depend on which history the model started from?

To answer this question, we introduce matched-history evaluation. We construct matched histories by taking two different histories and applying the same updates, so both end with the same facts, and then ask both the same downstream question. We also use two references, which are control runs of the same model. The first reference gives the model the final facts from the start, while the second reference applies the same updates to a history where those facts are already correct. We then require the model to correctly report every fact needed to answer the downstream question under each matched history. If the model answers the question correctly in both references, then the answer should no longer depend on which history the model started from. Thus, a difference in the downstream answers between matched histories shows an answer's dependence on history. We call this history dependence.

We find that the answer to the original question is a resounding yes. We set up a synthetic task

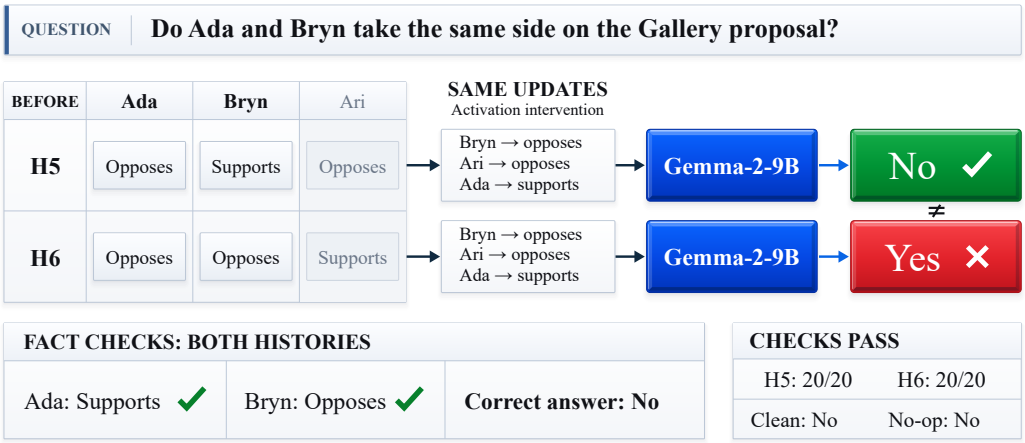


Figure 1: **Matched histories.** Two histories of the synthetic task, H_5 and H_6, receive the same updates and the same downstream question. The only thing that is different is the starting history, which here is each person's stance on the Gallery proposal before the updates, where "supports" means in favor. Gemma-2-9B reports both facts that the question needs correctly under both histories, and both references answer No. One reference is given the final facts from the start (clean) and the other has the same updates applied to facts that were already correct (no-op). The checks pass and the answers still differ, so the downstream answer depends on the history. The outdated facts imply No under H_5 and Yes under H_6, which we derive from those facts instead of measuring. Ari is not in the question. His starting stance differs between the two histories so that the same updates change exactly two facts under each, and both prompts have 455 tokens before the question.

which tracks whether people support or oppose various proposals, and find that history dependence recurs across both in-context updates and activation interventions in Gemma-2-9B and Qwen3-8B. On the public MQuAKE benchmark, it affects 24% of multi-hop questions in Qwen2.5-7B and 26% in Gemma-2-9B. The effect survives in the larger models tested, Gemma-2-27B and Qwen2.5-32B. In addition, we find that an outdated fact related to queried facts raises the error rate between 12.5 and 15.6 percentage points as opposed to an unrelated fact. This suggests the model is not simply influenced by earlier facts, but actively uses them.

We attempted to repair the model. We used activation patching on the KV cache to clean up the internal record by overwriting part of it with values from a history where the updated facts were already correct. We found that later layers carry the effect, but it is spread across many transformer blocks. A targeted cache refresh repaired only 43% of failures while introducing new errors. Given the limits encountered with internal repair, we prevent the use of outdated information by using an external deterministic system. We store facts in a dictionary outside the model. Each update request makes a new copy of the dictionary where the entry corresponding to the fact that is affected will be changed, and the earlier copies are kept but aren't read when answering questions. A trained interpreter's job is to turn the question into an operation for the dictionary. History dependence, by construction, is fully mitigated, with the repair answering 99% of MQuAKE cases correctly on average, versus 38% when given only corrected facts, demonstrating that removing outdated facts from the prompt is not enough.

To summarize, our contributions are as follows:

- **History dependence and a way to measure it.** We show that models which report every updated fact correctly can still answer differently depending on their starting history, and introduce matched-history evaluation to isolate this failure from other downstream errors. The effect persists across update methods, model families, a public benchmark and larger models. (§2–§3).
- **Internal repair is unreliable.** The effect is spread across many transformer blocks, and internal methods repair some failures while introducing more new failures than they fix. (§4).
- **External state removes it.** Keeping facts in an external dictionary, with a trained interpreter mapping each question to a dictionary operation, eliminates history dependence by construction. (§5).

2 Matched-history evaluation

Consider a record which says that Ada opposes a proposal, and an update which changes this fact so that Ada is now in favor. When asked directly, the model reports that Ada is in favor. However, this does not tell us whether the model uses the new fact, or whether its later answers still depend on what the record said before. We test this by comparing histories.

Each *fact* gives one value, such as Ada's stance on the proposal, and the model reads each fact as one statement. A *history* is the set of facts that a model starts with, in the order of their statements. An *update* sets one fact to a new value. After an update, we call the earlier value of that fact an *outdated fact*, and we call the facts after all updates the *final facts*. Two histories are *matched* when they receive the same updates and so end with the same final facts (Figure 1). A *downstream question* is a question that the model can only answer correctly by using an updated fact together with other facts. Matched histories have the same final facts, so they have the same correct answer. A *history-dependent answer* is an answer to a downstream question that still differs between two matched histories. Such a question is history dependent, and we call this failure *history dependence*.

Checks. An answer that changes with the history could have a simpler explanation than history dependence. We therefore only count a question when three checks pass, each ruling out one such explanation (Table 1). The first check asks for each fact that the question needs on its own, in a single-fact question, and the model must report it correctly under every history. The other two are control runs, which we call references, where the model must also answer the downstream question correctly (Appendix A). Thus, when a counted answer still changes with the history, none of these explanations can account for it.

Updates and tasks. We test two update methods. An *in-context update* adds a correction to the prompt (Zheng et al., 2023), and the outdated statement stays in the prompt. An *activation intervention* leaves the prompt as it is. Instead, a small trained function rewrites the model's activations at the statement of the updated fact, similar to representation finetuning (Wu et al., 2024). Neither method changes the model weights. Besides the public MQuAKE benchmark (§3.2), we use a synthetic task where every fact is new to the model. Each fact says that one person at a fictional community meeting is in favor of or opposed to one proposal, and a history is one record of these facts. We update three facts, each of which can start with either value, giving eight matched histories, H_0 to H_7. The updates change nothing under H_0, making it the second reference. A *case* is these eight histories with their questions, and each downstream question combines two of the updated facts.

Check	Explanation it rules out
Single-fact questions	The update failed
First reference: final facts given from the start	The model cannot answer the question even with correct facts
Second reference: the same updates, where they change nothing	The update method itself breaks the answer

Table 1: **What each check rules out.** A question is only counted when all three checks pass.

History	Facts changed	Tokens before Q	Answer
H_0	0	455	No
H_1	1	456	No
H_2	1	456	Yes
H_3	2	457	No
H_4	1	454	Yes
H_5	2	455	No
H_6	2	455	Yes
H_7	3	456	No

Table 2: **The worked example under all eight histories.** Every history passes every check, and the answer still changes with the history. The correct answer is No.

Worked example. Consider the question "Do Ada and Bryn take the same side on Gallery?" In the final facts Ada is in favor and Bryn is opposed, so the correct answer is No. Under every history Gemma-2-9B, updated with an activation intervention, passes every check. However, its answer changes with the history (Table 2). In fact, the model answers No under H_5 and Yes under H_6 (Figure 1), two histories where the updates change the same number of facts and the prompts have the same number of tokens. The two answers are not close either, with No at a probability of 0.78 under H_5 and 0.02 under H_6. Thus, the model has every fact that it needs, and its answer still follows the history.

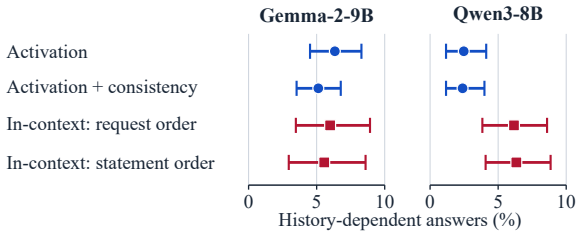


Figure 2: **Synthetic-task rates.** Rates are averages over 64 cases, and lines are 99.375% intervals, the level that corrects for the eight comparisons. Circles are activation interventions and squares are in-context updates.

Model	History dependent	Related vs. unrelated (pp)
Qwen2.5-7B	94 (24%)	+12.5
Gemma-2-9B	101 (26%)	+15.6
Qwen2.5-7B†	57 (15%)	+6.8
Qwen2.5-32B†	9 (2%)	+0.7
Gemma-2-27B	41 (11%)	+6.4

Table 3: **History dependence on MQuAKE.** Counts are history-dependent questions out of 384 per model. The last column is the gap in wrong answers between related and unrelated histories, in percentage points. †Measured again with one prompt format for both Qwen2.5 sizes (§3.3). Intervals are in Appendix C.

3 History dependence across settings

We find history dependence in every setting we test, under both update methods, in six models, on facts that we invented and on facts from a public benchmark. Our rates are out of the downstream questions we ask.

3.1 On a synthetic task with new facts

We run 64 cases of the synthetic task on Gemma-2-9B and Qwen3-8B, with two in-context updates, which differ in the order of their corrections, and two activation interventions, which differ in how they are trained (Appendix A). We find history-dependent answers in all eight combinations of model and update, in 2.4% to 6.3% of the questions (Figure 2), and every confidence interval excludes zero. This demonstrates that history dependence is not tied to one model or one update method.

History dependence survives stricter controls. After seeing these results we chose two stricter controls, fixed them, and ran 64 new cases on Gemma-2-9B with an activation intervention. The controls require the model to answer every other downstream question of the case correctly as well, and they only compare two histories where the updates change the same number of facts and the prompts have the same number of tokens. We count these results in cases, where a case counts when at least one of its questions is history dependent. We find that a history-dependent answer remains in 22% of the new cases. On the first 64 cases, pooling all eight combinations of model and update, the same controls leave a history-dependent answer in 29 cases, which is 45% of cases. These 29 cases hold 64 history-dependent answers, and 60 of them come from the activation interventions (Appendix B).

3.2 On public multi-hop questions (MQuAKE)

To test whether history dependence holds on questions we didn't write, we use 128 cases from MQuAKE-CF-3k-v2 (Zhong et al., 2023). Here a case is a chain of facts with updates and a multi-hop question in three phrasings, for 384 downstream questions per model. A multi-hop question follows the whole chain, for example from a person to his religion, to the founder of that religion and to the founder's workplace. We give the facts to the model in context and never edit the model weights, so these rates measure in-context updates rather than weight edits.

We give each case four matched histories. Each history states one fact about the entity that the question asks about, the queried entity, and one fact about an unrelated entity. In the two *related histories*, the fact about the queried entity is outdated, and it is stated either first or last. In the two *unrelated histories*, the fact about the unrelated entity is outdated instead. All four histories then receive the same corrections, which set both facts to their final values (Appendix C). In MQuAKE the outdated fact is often the real one, so these questions also test whether a correction in context overrides a belief that the model holds from pretraining.

About a quarter of questions are history dependent. We find that 24% of questions are history dependent in Qwen2.5-7B and 26% in Gemma-2-9B (Table 3). A question only counts when it passes every check, so these rates are a lower bound. In fact, among the questions that pass every check, the rates are 39% and 36% for Qwen2.5-7B and Gemma-2-9B.

The new fact is reported, and the outdated fact is used. In one case, where the question asks where the founder of William Levada's religion worked (Table 7 in Appendix C), Qwen2.5-7B reports all three final facts correctly under every history. It answers Edinburgh under three of the four histories, and Galilee under the related history that states the outdated religion first. Galilee is where the chain ended before the update. This demonstrates that a model can report the new fact and still answer from the outdated fact.

A related outdated fact does more harm. We find that under the related histories, Qwen2.5-7B answers wrongly 12.5 percentage points more often than under the unrelated histories, and Gemma-2-9B 15.6 points more often. Since the unrelated histories also contain an outdated statement in the same two positions, ordinary sensitivity to the prompt cannot explain this gap. This suggests that earlier facts are actively used by the model, as opposed to simply influencing it.

3.3 In larger models

We find that history dependence also appears in every larger model that we test. The MQuAKE results of §3.2 use a different prompt format, which is the fixed text around the facts and the question, for each model. We therefore measure Qwen2.5-7B and Qwen2.5-32B again with one shared prompt format. With the shared format, history dependence affects 15% of questions in Qwen2.5-7B, versus 24% under the format of §3.2, and 2% in Qwen2.5-32B, with a confidence interval that excludes zero (Table 3, Appendix C). Gemma-2-27B, measured with its own prompt format, has 11%. On the synthetic task the larger model has the higher rate, at 11% for Qwen3-32B and 6% for Qwen3-8B (Appendix B). Thus, across both update methods, both tasks and all six models, a model that passes every check can still answer from its history.

4 Internal repair is unreliable

An *internal repair* changes something inside the model so that the outdated facts no longer reach the answer. Naturally, we first looked for a single part of the model that carries history dependence, so that we could fix it. We found that the later layers carry history dependence, but that it can't be removed from those layers cleanly. We tested Gemma-2-9B on the synthetic task, updated with the first of the two activation interventions.

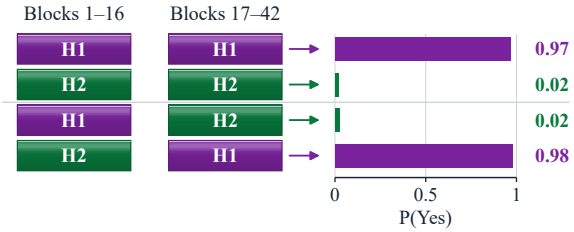


Figure 3: **Cache crossover.** One of the six cases, where each row combines cache blocks from its two histories, here called H_1 and H_2, and the answer follows the history that supplied blocks 17–42.

Gemma-2-9B has 42 transformer blocks, which we also call layers. The model stores the context that it has read in the KV cache, as cached keys and cached values for each block. A later question reads the context only through the cache, so we could change the cache before asking a question (Appendix D).

4.1 Later layers carry history dependence

We selected six cases, and for each we took one downstream question and two matched histories that give different answers to it. We built a cache with blocks 1 to 16 from one history and the remaining 26 blocks from the other. In this cache crossover, we found that in five of the six cases (Figure 3) the answer followed the history that supplied the later blocks, while the model still reported every fact correctly.

No small part carries history dependence. We then found, in a seventh case, that copying only what the cache holds for blocks 17 to 24 was enough to move the answer from one history to the other. However, no smaller part we tried was enough. We copied the part of one block that serves one pair of attention heads, one such part at a time, and none of these 64 copies moved the answer (Appendix D).

4.2 A cache refresh creates new history dependence

Since there was no small part to fix, we tried a repair over all of blocks 17 to 24 at once, which we call a *cache refresh*. Before asking a question, we overwrote the cached values of these blocks with the cached values from H_0, the history in which the updated facts already have their final values. We tested the cache refresh on 256 new cases with 83 history-dependent questions.

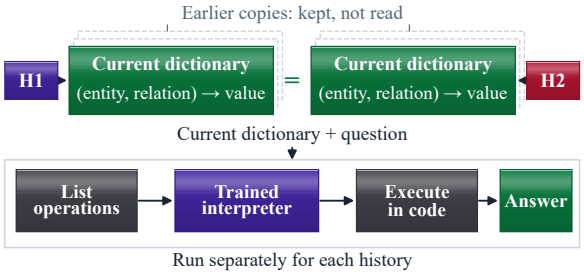


Figure 4: **External-state repair.** Two histories that end with the same final facts, labelled H_1 and H_2 in the figure, have the same current dictionary, so the interpreter reads the same input and code runs the same operation.

Answered by	Repaired (of 83)	Errors left
Gemma-2-9B alone	0	3,490
Cache refresh	36	2,893
External-state repair	83	0
Gemma-2-9B's reported facts + code	83	584

Table 4: **Four ways of answering on the same 256 cases.** Errors are all wrong answers to downstream questions over all histories, whether or not the answers are history dependent. The external-state repair is the simpler version of this section, with Qwen2.5-7B as its interpreter.

The refresh creates more history dependence than it repairs. The refresh repaired only 36 of the 83 questions, or 43%, where a repaired question is answered correctly under every history. At the same time, it made 40 other questions history dependent. Running the same refresh on blocks 25 to 32 repaired just 2% (Appendix D). We also note that the refresh requires the cache of H_0, which is a history that is already correct. However, correct facts are what an update is meant to produce. Thus, the cache refresh is not a reliable repair.

5 External state removes history dependence

Given the limits encountered with internal repair, we take the facts out of the model and keep them outside as *external state*. We find that external state removes history dependence. This is because the component that produces the answer never reads the history. Storing the facts externally is not enough without this.

5.1 Design

We store the facts in a dictionary outside the model, where each entry maps an entity and a relation to a value. Ordinary program code fills the dictionary from the facts of the history. Each update makes a new copy of the dictionary where only the entry of the updated fact is changed, and the earlier copies are kept but aren't read when answering questions (Figure 4). To answer a question, an *interpreter* reads the current dictionary and the question, and chooses an *operation* among those that the dictionary allows. An operation starts at an entity and follows up to four relations. Code then runs the operation and returns the value where the operation ends. For the example of §3.2, the operation starts at William Levada, follows the three relations of the chain and ends at Edinburgh. If no operation answers the question, the interpreter refuses to answer. The interpreter is a language model with a small trained adapter added for MQuAKE (§5.3). In this design, it never writes the final answer. We call the dictionary, the interpreter and the code together the *external-state repair*.

Why the history cannot matter. Two matched histories end with the same final facts, so they produce the same current dictionary. This means the interpreter is given the same input under both histories, and so it chooses the same operation. History dependence, by construction, is fully mitigated, leaving only the question of whether the interpreter chooses the right operation.

5.2 The synthetic task is fully repaired

We first test a simpler version of this design on the same 256 cases as the cache refresh. In this version Qwen2.5-7B, without any training, fills the dictionary and writes the operation itself (Appendix E). The interpreter doesn't have to be Gemma-2-9B, the model we updated, because the facts are now in the dictionary. We find that every question is then answered correctly under all eight histories (Table 4), so all 83 history-dependent questions are repaired with no errors left, while the cache refresh repaired 36 and left 2,893 errors.

The facts Gemma-2-9B reports repair the 83 questions too. In a second test with no dictionary, code combines the single facts that Gemma-2-9B itself reports, and all 83 questions are repaired as well. This is what our checks predict, since they already required these facts to be reported correctly. Thus, for these questions the facts Gemma-2-9B reports are correct, and what fails is the way it uses them.

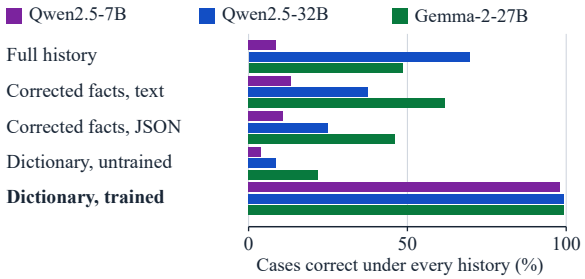


Figure 5: **Repair results.** The trained interpreters are correct in nearly every case, and the three base models given only the final facts as text are not. Cases are out of 128 per model. Bars follow the legend order, and the trained bars are the mean of two seeds.

5.3 MQuAKE cases are answered correctly

MQuAKE questions are harder to turn into operations, and untrained Qwen2.5-7B as the interpreter answers only 35% of them correctly on a sample of 16 cases (Appendix E). We therefore train one interpreter for each of Qwen2.5-7B, Qwen2.5-32B and Gemma-2-27B. Each interpreter is the frozen base model with a LoRA adapter (Hu et al., 2022), trained on 1,024 other MQuAKE cases with two training seeds (Appendix F). None of the multi-hop questions we test on appear in the interpreters' training data. We evaluate on the 128 MQuAKE cases and four histories of §3.2, and we count a case as correct only if its three multi-hop questions and its single-fact questions are all answered correctly under all four histories.

99% of cases, versus 38% from corrected facts. We find that the trained interpreters are correct in 99% of cases on average (Figure 5). As a baseline we give each of the three base models only the final facts as text, which we call the corrected facts, with no history and no dictionary. The baseline models are correct in 38% of cases on average, and the gain of the interpreter has a confidence interval well above zero for every model. The interpreters are trained on other cases of the benchmark and the baseline models are not, so the gain compares two whole systems. Even the best baseline model, Gemma-2-27B, is correct in only 62% of cases with nothing but the corrected facts in its prompt. This demonstrates that removing the outdated facts from the prompt is not enough.

Every history-dependent question is repaired. Table 3 counts 94 history-dependent questions for Qwen2.5-7B, 9 for Qwen2.5-32B under the shared prompt format and 41 for Gemma-2-27B, and the trained interpreters answer all of them correctly under every history.

What the interpreter reads	Cases correct
Dictionary only	98%
Dictionary and final facts as text	90%
Dictionary and history	56%

Table 5: **Letting the history back in.** The dictionary and the code are the same in every row. Rows are the mean of two training seeds for Qwen2.5-7B, and Appendix G gives each seed.

5.4 The history has to be kept out

Showing the interpreter the history breaks the repair. The dictionary holds the facts, but history dependence is removed because the interpreter never reads the history. We test this claim with the trained Qwen2.5-7B interpreter on MQuAKE, where we keep the dictionary and the code unchanged and add the history, with its corrections, to the input of the interpreter (Table 5).

We find that the share of correct cases falls from 98% to 56%. As a control for the longer input, we add the final facts as text instead of the history, and 90% of cases stay correct (Table 12 in Appendix G). Thus, once the interpreter reads the outdated facts, it refuses questions that it answered correctly before, with the same dictionary, and every new error is such a refusal (Appendix G).

The interpreter can vary. We also train two other interpreters from Qwen2.5-7B on the same dictionary, one that writes the operation itself for code to run and one that writes the answer itself. These two are correct in 98% and 97% of cases, compared to 98% for the interpreter that chooses an operation, and each of the three gives the same answer under all four histories (Table 13). Together with Table 5, these results demonstrate that the repair comes from keeping the history out, and that the choice of interpreter is free to vary.

6 Related work

Updating models and checking the update. Knowledge editing changes model weights to insert a new fact (Mitchell et al., 2022; Meng et al., 2022, 2023), while in-context editing supplies the new fact in the prompt (Zheng et al., 2023). Both kinds of editing are usually checked by asking for the edited fact, its paraphrases and unrelated facts. MQuAKE (Zhong et al., 2023) and RippleEd-

its (Cohen et al., 2024) show that multi-hop and related questions often fail after an edit that passes this check. Later work attributes some of these failures to shortcuts (Ju et al., 2024) and documents other side effects of editing (Li et al., 2024; Gu et al., 2026). However, these evaluations do not distinguish where a failure comes from. We isolate the failures caused by outdated facts using matched histories, where the final facts are held fixed and only the history varies.

Outdated information. Wang and Sun (2025) show that earlier values of a repeatedly updated key interfere with retrieving the latest value. We instead require the latest value to be retrieved correctly, and find that later answers can still depend on the earlier values. Vafa et al. (2024) test whether a model treats different sequences that reach the same state in the same way, and we apply this test to an update.

Changing activations and the cache. Activation steering and representation engineering change model behavior by editing activations instead of weights (Turner et al., 2023; Zou et al., 2023; Li et al., 2023), and ReFT, which our activation interventions resemble, trains such an edit within a low-rank subspace (Wu et al., 2024). Li (2026) treat the KV cache itself as something that can be edited and composed. They report that a model answering directly keeps acting on the old value after its cached keys and values are overwritten, and our cache refresh reaches a similar conclusion.

External memory. For multi-hop questions under edits, MeLLo (Zhong et al., 2023) stores the edited facts externally and prompts the model to check its answers against these facts, and GMeLLo (Chen et al., 2024) combines the model with a knowledge graph. These methods are the closest to our repair. We ran our own untrained versions of GMeLLo, where the two main ones differ in how many facts they retrieve when the graph query returns no answer. These answer 71% and 70% of our MQuAKE cases correctly, versus 98% for the trained Qwen2.5-7B interpreter (Appendix G). Unlike these methods, we ask whether the answer still depends on the history once the facts are outside the model.

7 Conclusion

Reporting an updated fact correctly does not show that the model uses that fact. With matched histories, we show that its downstream answers can still depend on its history. We attempted to repair this inside the model and found that the later transformer blocks carry history dependence, but our cache refresh of blocks 17 to 24 created more history-dependent questions than it repaired. Instead, moving the facts outside of the model worked. An external dictionary with a trained interpreter that never reads the history removes history dependence by construction, while answering 99% of MQuAKE cases correctly. Our results demonstrate that knowing the new fact is not enough, as the outdated fact still reaches the answer. Thus, for updates which need to be reliable, the facts should be kept as external state and the history kept away from the interpreter.

We leave open why the model keeps using the outdated facts, and we see this as a question for AI safety as well. When an update is meant to remove a harmful belief, a model that reports the correction without using it would still pass a check of its outputs. In future work, we plan to use matched histories to test this directly on beliefs that models are known to hold from pretraining.

Limitations

What we measure. Our synthetic task uses fictional records, which means that we can't say how often history dependence happens in real use. On MQuAKE, the facts are given to the model in context. Our results therefore say nothing direct about methods that edit weights, and we don't know how much of MQuAKE the models saw in pretraining. We also found that the number of history-dependent questions for one model changes with the prompt format. For Qwen2.5-7B, the counts are 94 and 57 in Table 3, and 32 when Qwen2.5-7B answers directly from the history with the prompt format of the repair evaluation (Appendix F). We compare two sizes each of Qwen2.5, Gemma-2 and Qwen3, where the two Gemma-2 sizes were not run under one prompt format. We only count a question when every check passes, so our counts miss some of the answers that the history affects.

Internal repair. We test internal repair on a single model, Gemma-2-9B, with the first of the two activation interventions only. We selected the cache crossover cases because their two histories give different answers, and we ran the search for a single part in one further case only. The cache refresh lowers the total number of errors by 0.8 percentage points, while it makes more questions history de-

pendent than it repairs. Our results show that the repairs we tried are unreliable, and they do not rule out that another internal repair could work.

What the external-state repair does. The external-state repair leaves the updated model as it is, so why the model keeps using the outdated fact is still an open question. History dependence is removed, but errors can remain, since the interpreter may refuse or choose a wrong operation, which code will then run. Our gains over corrected facts compare whole systems, and we did not separate the gain of the dictionary from that of the training or of having code run the operation. We only trained the two other interpreters of §5.4 for Qwen2.5-7B. Both of them read the dictionary that our design builds, which means they aren't separate solutions to the whole problem of updating. For MeLLo and GMeLLo, we ran our own untrained versions with Qwen2.5-7B, and these are not reproductions of the published systems (Appendix G).

How the external-state repair is evaluated. We measure history dependence on the same 128 MQuAKE cases on which we evaluate the repair, and we had already inspected these cases before we built the repair. We train the interpreter on other cases from the same benchmark. These training cases share relations and entities with the cases that we test on, and some of the single-fact questions overlap with the training questions of the interpreter. Since code fills the dictionary by reading known sentence templates, we leave open whether updates written in free text can be stored correctly. We used two training seeds per model, which is too few to describe the variance between seeds, and this variance is large when the interpreter reads the history. On the synthetic task, the external-state repair was tested with an untrained interpreter that writes the operation directly, an earlier version of the design than the one we train for MQuAKE. To fill the dictionary, this version is given the statements and updates of each history as text, which the cache refresh does not use. We also knew the question forms of the synthetic task when writing the instructions of the untrained interpreter. The untrained interpreter repairs all 83 questions on the synthetic task. This largely follows from our checks, which already required the facts behind those questions to be reported correctly. The same holds for the repair from the facts that Gemma-2-9B itself reports.

Ethical considerations

The records in our synthetic task are fictional. The MQuAKE cases contain counterfactual statements about real people and organizations, such as the example in §3.2. These are stipulations of the benchmark, and we make no claims about those people.

References

Ruirui Chen, Weifeng Jiang, Chengwei Qin, Ishaan Singh Rawal, Cheston Tan, Dongkyu Choi, Bo Xiong, and Bo Ai. 2024. LLM-based multi-hop question answering with knowledge graph integration in evolving environments. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 14438–14451. Association for Computational Linguistics.

Roi Cohen, Eden Biran, Ori Yoran, Amir Globerson, and Mor Geva. 2024. Evaluating the ripple effects of knowledge editing in language models. *Transactions of the Association for Computational Linguistics*, 12:283–298.

Xiaojie Gu, Ziying Huang, Weicong Hong, Jian Xie, Renze Lou, and Kai Zhang. 2026. The model agreed, but didn't learn: Diagnosing surface compliance in large language models. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 2514–2529. Association for Computational Linguistics.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Tianjie Ju, Yijin Chen, Xinwei Yuan, Zhuosheng Zhang, Wei Du, Yubin Zheng, and Gongshen Liu. 2024. Investigating multi-hop factual shortcuts in knowledge editing of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8987–9001. Association for Computational Linguistics.

Bojie Li. 2026. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. 2023. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems*, volume 36, pages 41451–41530.

Zhoubo Li, Ningyu Zhang, Yunzhi Yao, Mengru Wang, Xi Chen, and Huajun Chen. 2024. Unveiling the pitfalls of knowledge editing for large language models. In *International Conference on Learning Representations*.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372.

Kevin Meng, Arnab Sen Sharma, Alex J. Andonian, Yonatan Belinkov, and David Bau. 2023. Mass-editing memory in a transformer. In *International Conference on Learning Representations*.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. 2022. Fast model editing at scale. In *International Conference on Learning Representations*.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. 2023. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*.

Keyon Vafa, Justin Y. Chen, Ashesh Rambachan, Jon Kleinberg, and Sendhil Mullainathan. 2024. Evaluating the world model implicit in a generative model. In *Advances in Neural Information Processing Systems*, volume 37, pages 26941–26975.

Chupei Wang and Jiaqiu Vince Sun. 2025. Unable to forget: Proactive interference reveals working memory limits in LLMs beyond context length. *arXiv preprint arXiv:2506.08184*.

Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. 2024. ReFT: Representation finetuning for language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 63908–63962.

Ce Zheng, Lei Li, Qingxiu Dong, Yuxuan Fan, Zhiyong Wu, Jingjing Xu, and Baobao Chang. 2023. Can we edit factual knowledge by in-context learning? In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4862–4876. Association for Computational Linguistics.

Zexuan Zhong, Zhengxuan Wu, Christopher D. Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15686–15702. Association for Computational Linguistics.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, and 2 others. 2023. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*.

A Matched-history details

Prompt. This appendix gives the full procedure of §2. Every prompt starts with the same two solved demonstrations, regardless of the history or the question. The record is placed after the demonstrations and has one sentence per fact, for example "Bryn is in favor of the Gallery proposal." The question and a line naming the two allowed answers are placed at the end. In the worked example, there are twelve facts from four people and three proposals. The facts we update are the Gallery stances of Bryn, Ari and Ada. Table 2 lists the eight histories.

In-context updates. A correction is appended for each request before the end of the record, for example "Correction: the record for Bryn on the Gallery proposal is updated. Bryn is now opposed to the Gallery proposal." The original statements are not removed. A correction is appended even when the fact is already correct. The two in-context updates differ only in the order of their corrections, which follows either the order of the requests or the order of the statements in the record.

Activation interventions. We number transformer blocks from 1. The first activation intervention acts on the output of block 16 of Gemma-2-9B, which has 42 blocks, and only at the token positions of the sentence being updated. For each of those tokens, we split the activation into the part inside 16 learned directions and the part outside of them. We keep the outside part, and predict a replacement for the inside part from what was kept and its average over the sentence. This prediction takes one matrix multiplication and a bias. We learn one such rule per requested value, for 286,752 trained numbers in total, and the model weights stay frozen. We trained on 64 pairs of starting records which differ in one fact, using the negative log likelihood of the correct answer to questions asked after the intervention as the loss, along with a small penalty of 0.01 on the size of the change. To choose which saved version to keep, we used a further 32 pairs. Training was started from an earlier trained version of the intervention. For the second activation intervention, we replace the same 16 directions but let the prediction also read the part being replaced, and we add to the loss a symmetric KL term between the answers given from the two starting records. Qwen3-8B has 36 blocks, and its intervention acts

on the output of block 14. We trained every intervention with two seeds, using records that are not used for testing, and froze it before testing.

Questions. Each history is asked 17 questions in two answer formats, for 34 in total. Of the 17, six ask whether a person favors a proposal, two ask whether a person opposes one, and nine are combined questions. The combined questions cover three pairs of people and three forms – whether both take the same side, whether both are in favor, and whether at least one is in favor. One answer format uses the words No and Yes, and the other uses letters. The answer is whichever allowed answer has the higher next-token probability, and no text is generated. A tie cannot count as an answer. Once the updates are applied, each question is asked on its own copy of the model's cached state, the KV cache, so that no question sees another question or its answer. The nine combined questions in both answer formats are the 18 scheduled questions of a case, and the rates on this task are out of them.

Checks for one question. Under our stricter controls, the other combined questions must be answered correctly as well, and we only compare histories that changed the same number of facts and have the same number of tokens before the question. For one combined question, this means each history has to pass 12 single-fact questions, which are six facts in two answer formats, along with the 8 other combined questions in the same format. That is 20 checks per history and 160 over the eight histories, with the two references being separate from these. All 160 pass in the worked example (Table 2). Rescaled over the two allowed answers, H_5 gives No a probability of 0.78, while H_6 gives it 0.02.

B Synthetic task details

The rates behind Figure 2 are in Table 6. For the activation interventions, we average the two seeds within a case, and count a case as affected if either seed has a history-dependent answer. Intervals come from resampling the 64 cases 10,000 times. For Gemma-2-9B, seed 0 of the second activation intervention uses the weights saved before the consistency stage of training, which is what our selection rule chose.

Stricter controls. We chose the stricter controls after seeing the first results, and then fixed them before running the 64 new cases of §3.1, which use Gemma-2-9B with both seeds of the first activation intervention and no retraining. Over both models, the four update methods and both seeds, there are 653 history-dependent answers before any stricter control. Requiring the two histories to change the same number of facts and have the same number of tokens leaves 430. Requiring the other combined questions to be correct as well leaves 64 of the 13,824 scheduled questions, in 29 of the 64 cases. Of these, 60 come from the activation interventions and 4 from the in-context updates. On the 64 new cases, 14 have a history-dependent answer, and the rate of 0.95% of scheduled questions has a 95% interval of 0.52% to 1.43%. The worked example of §2 is one of them.

Model	Update	Rate	Interval	Cases
Gemma-2-9B	Activation	6.34	[4.51, 8.29]	50
	Activation + consistency	5.12	[3.53, 6.77]	41
	In-context, requests	5.99	[3.47, 8.92]	30
	In-context, statements	5.56	[2.95, 8.59]	25
Qwen3-8B	Activation	2.47	[1.17, 4.12]	24
	Activation + consistency	2.39	[1.17, 3.99]	25
	In-context, requests	6.16	[3.84, 8.59]	38
	In-context, statements	6.34	[4.08, 8.85]	40

Table 6: **Synthetic task results.** Rate is the percent of scheduled questions with a history-dependent answer, with its 99.375% interval. Cases is the number of the 64 cases that are affected.

A larger model on the same cases. We also ran Qwen3-32B on the same 64 cases with the two in-context updates. History-dependent answers appear in 128 of 1,152 scheduled questions (11.1%) when corrections follow the request order, and in 123 of 1,152 (10.7%) when they follow the statement order. These are the same cases as before, so this adds a model, not new cases.

C MQuAKE details

Histories and checks. The two related histories place the outdated fact about the queried entity either first or last around the records for the rest of the chain, and the two unrelated histories do the same with the matched outdated fact about an unrelated entity. We ask for the facts needed by a question independently, without giving their answers to the multi-hop question. Answers are read from the first

line of the response and matched against the aliases of the entities, without using the correct answer to decide between readings. We count a question when two of its histories give different answers and only one of those answers is correct, where every question we count is also answered correctly from the final facts alone. Note that this is not the standard closed-book MQuAKE setting, since we give the facts in context and never edit the weights.

Long chains. For the 195 questions with three or four hops, we also check that the model can answer the rest of the chain when starting from the updated fact. History dependence remains even then, affecting 16% of them in Qwen2.5-7B, 33% in Gemma-2-9B and 17% in Gemma-2-27B, which are 32, 65 and 34 questions.

Intervals. We compute intervals at a 98.75% level by resampling whole cases 10,000 times, keeping the three phrasings of a case together. For history dependence, this gives 17.7%–31.8% for Qwen2.5-7B and 18.6%–34.4% for Gemma-2-9B. For the related-versus-unrelated gap, the intervals are 8.3 to 17.4 percentage points for Qwen2.5-7B and 10.8 to 21.0 for Gemma-2-9B. We also report intervals under the shared prompt format of §3.3. There, history dependence has an interval of 9.4%–20.8% for Qwen2.5-7B and 0.3%–5.5% for Qwen2.5-32B, while the gaps have intervals of 3.9 to 10.2 and -0.5 to 2.3 percentage points. Restricting to questions that pass every check under that format gives rates of 24% and 4%. Note that the gap is computed over all 384 questions, where any question failing the checks counts as zero. We report the gap of Gemma-2-27B without an interval, and its 41 history-dependent questions come from 26 cases.

Sensitivity. With a stricter reading of the answers that only accepts the canonical name of an entity, the counts are 87 of 384 for Qwen2.5-7B and 98 of 384 for Gemma-2-9B. We repeated 12 complete cases per model in a fresh process, and all 12 gave the same answers.

The worked example. Table 7 gives the answers of both models under each history and both references. Both models fail under exactly one history, but not the same one.

Outdated fact	Qwen2.5-7B	Gemma-2-9B
Related, first	Galilee	Edinburgh
Related, last	Edinburgh	Galilee
Unrelated, first	Edinburgh	Edinburgh
Unrelated, last	Edinburgh	Edinburgh
Facts from the start	Edinburgh	Edinburgh
Already correct	Edinburgh	Edinburgh

Table 7: **The MQuAKE worked example.** The correct answer is Edinburgh. The last two rows are the two references.

D Internal repair details

Cache crossover. We used the following criteria for §4.1 and Figure 3, where p is the probability of Yes over the two allowed answers. The two original histories must differ in p by at least 0.10. The answer then follows the later blocks if the two mixed caches are, on average, within 0.05 of the history that supplied their later blocks and at least 0.10 away from the history that supplied their earlier blocks. We also required each direct fact to stay correct with a probability of at least 0.95 and a change of at most 0.02. In the second answer format, one of the six cases shows the same pattern, while two are mixed and three have too little difference between their two histories to tell. The seventh case has 517 tokens before the question and changes two facts in both histories. A site is one block and one group of keys and values, which serves two attention heads. Figures 6 and 7 show all 64 sites. Copying only the values stored for blocks 17 to 24, at the last 91 token positions of the context, transfers the answer in both directions and in both answer formats. The best one, two, four and eight sites transfer zero, one, two and three of the four answers. On two other cases, the same region transfers only two and one of the four. In the seventh case, copying the full activations after block 16 at 30 of those positions also transfers all four answers, but this replaces everything the later blocks read there.

From the cache to the answer. All of the following is measured on the seventh case, in four conditions (two directions and two answer formats). Blocking the outputs of all 128 heads in blocks 17 to 24 at the question positions transfers none of the four answers, and restoring them transfers all four, with all eight direct checks intact in both. Among single heads, we ranked the 128 by the smaller of what blocking removes and what restoring supplies,

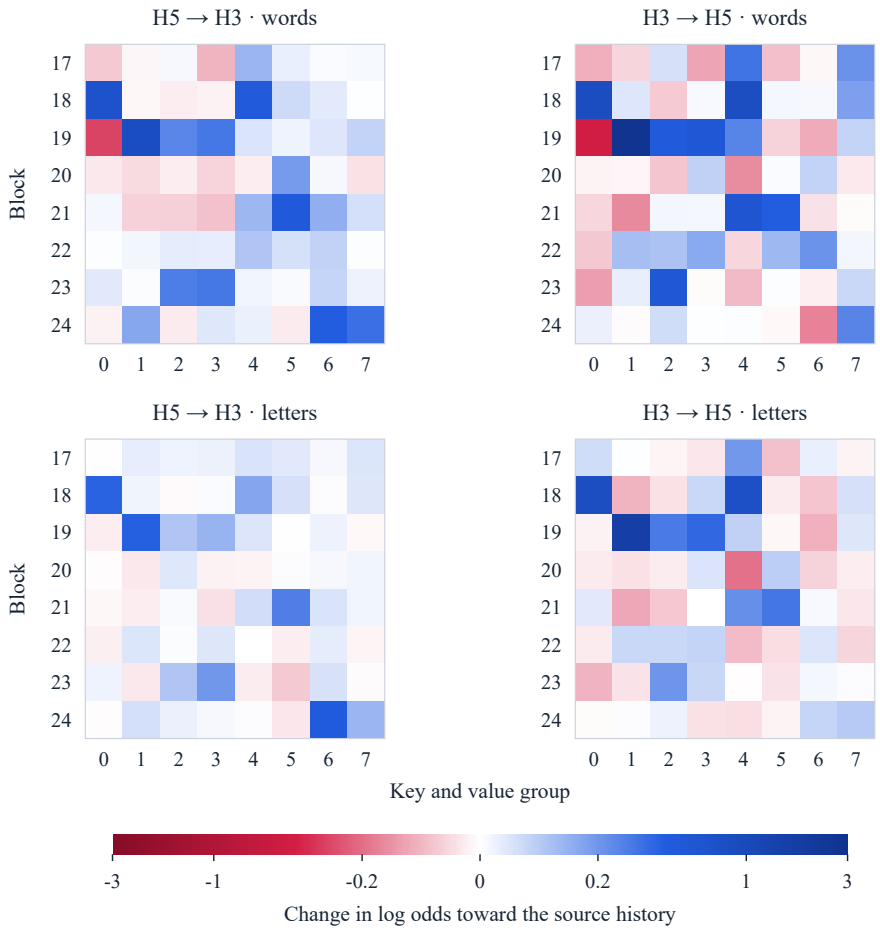


Figure 6: **Copying one site at a time.** Each cell is one copy in the seventh case, of the values that the KV cache stores for one block and one group of keys and values. Arrows go from the source history to the receiving history, and the panels differ in answer format (words or letters). Color is the change in log odds of the answer toward the source history, linear within ± 0.2 and logarithmic outside it. No single site out of the 64 transfers the answer in all four panels. Red is negative, white is zero and blue is positive, with limits of ± 3 for both heatmaps.

and fixed the best one, two, four and eight before testing them in the other format and direction. The best eight transfer none of the four answers when blocked and all four when restored. Blocking these eight heads removes anywhere between 71% and 95% of the effect, while restoring them supplies between 75% and 89%. Their score on our measure is 0.78, or 78% of the effect, meaning they do not account for all of it. On the writing side, the attention output of block 17 alone transfers none of the four answers.

Cache refresh. Table 8 gives the full accounting for the 256 new cases of §4.2, which we ran with both seeds for 9,216 scheduled questions. Over all 73,728 answers to combined questions, we find that the refresh corrects 944 errors while introducing 347 new ones. On direct fact questions, it corrects 84 and introduces 76. We only count a repair when the question becomes correct under all eight histories and all of its checks stay correct. Before the refresh, 83 questions were history dependent. Of these, 36 are repaired, 35 stay history dependent, and 12 are no longer counted without having been repaired. The values are copied at the trailing positions of the context, aligned from its end. Figure 8 gives the intervals for these counts.

E External state on the synthetic task

The 256 cases. Table 4 compares four ways of answering on the 256 cases of §4.2. The cache refresh makes 347 answers wrong that the model had right, combining the model's own answers makes 346 wrong, and the external dictionary makes none wrong. Combining the model's own answers uses its saved direct answers under the same history, seed and answer format, with the operation sup-

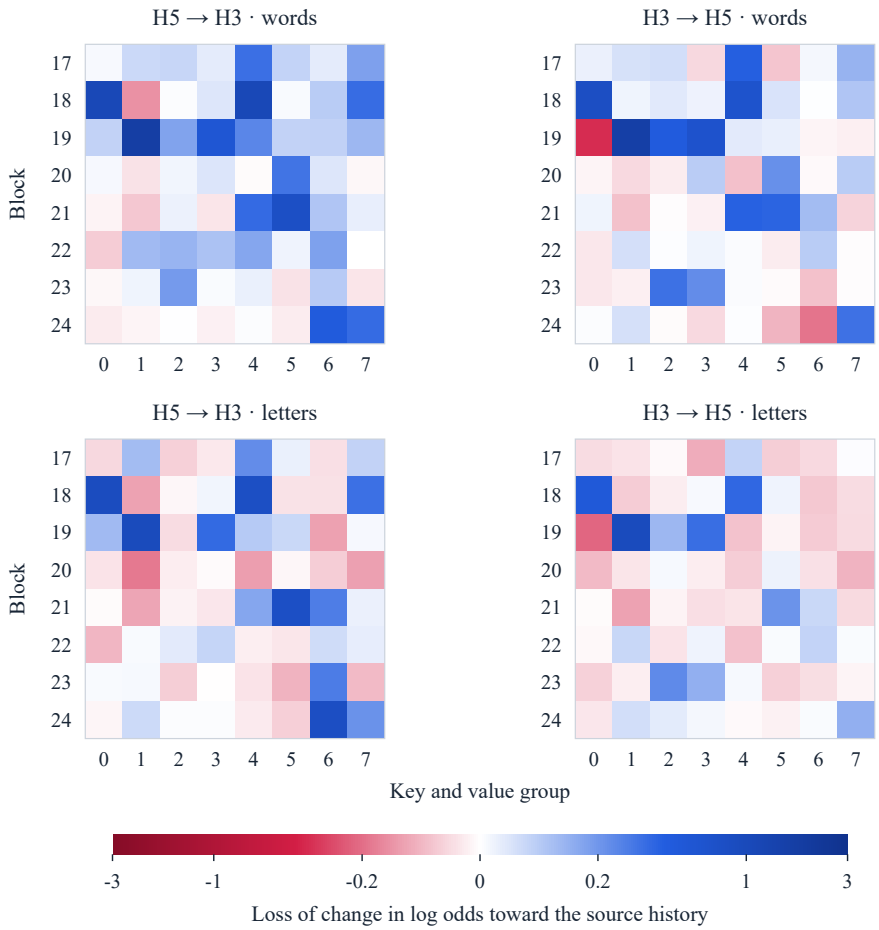


Figure 7: **Leaving one site out.** The same case, sites and panels as Figure 6. Each cell copies all of blocks 17–24 except for one site, which keeps its own values. Color is how much of the change in log odds from copying the whole region is lost, on the same scale.

plied by us, where an invalid direct answer is never replaced. It leaves the 312 errors on direct questions unchanged, corrects 3,252 errors on combined questions and introduces 346. We scored the external dictionary against the same 83 questions for each seed, which are 37 for seed 0 and 46 for seed 1. In every case, all 34 questions were answered correctly under all eight histories, for 69,632 answers. We used Qwen2.5-7B both to read each record into a dictionary entry and to write the operation for each question directly, where it sees the question but never the history. Its records are read from the original text, 641 distinct lines without an error, while the update requests are given as in the original task. It turns every one of the 5,984 distinct questions into the right operation.

New cases. On 64 new cases with eight histories and 38 questions each, the external dictionary answers all 19,456 questions correctly. The model answering from the full history is correct on 79.8%, and a common alternative, where it is the model that picks the facts it needs out of the full history and code that solves the rest, is correct on 64.8% and on none of the 64 cases in full. Its answer changes with the history in 1,335 of the 2,176 questions about the final facts, which are 34 of the 38 in each case. With Gemma-2-9B in place of Qwen2.5-7B, the external dictionary is correct on all 4,864 questions of the first 16 of these cases.

Letting the history back in. On the synthetic task we repeat the test of §5.4 with the untrained Qwen2.5-7B interpreter of §5.2. This test uses 16 cases with 4,352 answers, where the final facts and the question are the same and only the outdated facts differ. Every answer is correct when the history is kept out. When we show the history while the model writes the operation, only 86.95% are correct, which is a drop of 13.05 percentage points with a 95% interval of 9.72 to 16.66. Here the answer changes with the history in 68 of 544

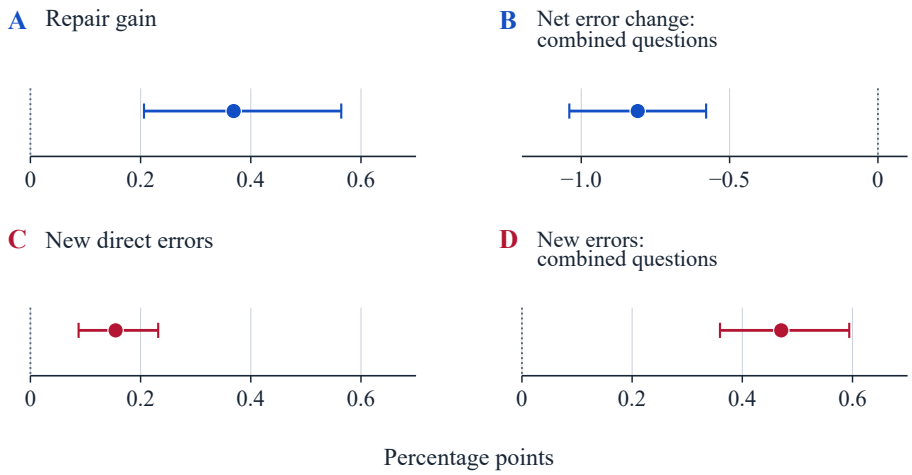


Figure 8: **Cache refresh.** Points are estimates with 98.75% intervals, in percentage points. (A) Repairs from refreshing blocks 17–24 minus repairs from blocks 25–32, over all 9,216 scheduled questions. (B) Change in errors on combined questions. (C, D) New errors on direct fact questions and on combined questions.

Cases / seeds	256 / 2
Scheduled questions	9,216
History dependent before	83
repaired (blocks 17–24)	36
repaired (blocks 25–32)	2
still history dependent	35
no longer counted	12
Newly history dependent after	40
History dependent after	75
Combined-question answers	73,728
errors before / after	3,490 / 2,893
corrected / introduced	944 / 347
Direct-fact errors	
corrected / introduced	84 / 76

Table 8: **Cache refresh accounting.** The history that supplies the refreshed values, where the facts were already correct, makes 2,000 errors on the same answers.

questions. When the model instead restates the computed answer, 92.16% are correct and 341 answers changed, which is a drop of 7.84 with an interval of 4.53 to 11.26.

Other kinds of question. On questions over a synthetic graph of relations this version is correct on 92.97%, where every miss is a rejected record and no answer is wrong. On 16 MQuAKE cases it is correct on 35.42%. Of its 48 distinct questions, 21 become the right operation. Of the rest, 14 follow the wrong number of relations, 11 use a wrong relation and 2 fail in other ways.

F External-state repair details

Training. Each interpreter is trained on 12,288 pairs of a question and a candidate operation, which we build from 1,024 MQuAKE-CF cases that are not among the 128 we test on. The adapters use rank 16 and alpha 32 with no dropout, on the query, key, value and output projections. Training runs for 768 steps. We use AdamW with a learning rate of 2e-4, an effective batch size of 16, 39 warmup steps and a cosine schedule, and take the checkpoint at the last step. The 12,288 training pairs contain 7,507 distinct ones. The loss is only on the Yes or No response. The same pairs and the same order are used for all three models.

Choosing an operation. Code first lists the operations that the current dictionary allows, with at most 64 candidates listed for a question. Each operation starts at an entity and follows up to four relations from it. We show the interpreter the current entries and the question along with one candidate at a time, and its job is to score whether that candidate answers the question. This score is the interpreter's logit for Yes minus its logit for No, and we choose the candidate with the highest score above zero. The system refuses to answer if no candidate has a score above zero. To check that equal dictionaries give equal choices, we repeated a set of questions without any caching, and no choice changed.

Results. The 901 questions are the 384 multi-hop questions and 517 check questions, which ask about single facts, about the unrelated entity and about the rest of the chain. We compare the trained interpreter with three alternatives. In the first, the model answers directly from the full history. In the second, the model is given only the corrected facts, as text or as JSON. In the third, the base

Method	Qwen 2.5-7B	Qwen 2.5-32B	Gemma 2-27B
Full history	11	89	62
Note before question	12	–	–
Corrected facts, text	17	48	79
Corrected facts, JSON	14	32	59
Dictionary, untrained	5	11	28
Dictionary, trained	124	128	127
second seed	127	126	127

Table 9: **Cases with every question correct under all four histories**, out of 128. The trained interpreter is given for both seeds, and the untrained one is the base model. The full-history prompt is not the same across the three models.

Baseline	Gain	Interval
Qwen2.5-7B		
Full history	89.5	[83.2, 94.9]
Note before the question	88.7	[82.0, 94.5]
Corrected facts, text	84.8	[77.3, 91.4]
Qwen2.5-32B		
Full history	29.7	[21.1, 39.1]
Corrected facts, text	61.7	[52.0, 71.1]
Gemma-2-27B		
Full history	50.8	[41.0, 60.5]
Corrected facts, text	37.5	[28.1, 47.3]

Table 10: **Gain of the trained interpreter** in percentage points of cases, with 97.5% intervals from 10,000 resamples of the 128 cases.

model acts as an untrained interpreter on the dictionary. When given only the corrected facts as text, Qwen2.5-7B is correct in 13% of cases, Qwen2.5-32B in 38% and Gemma-2-27B in 62%. Table 9 has every condition, and also includes one for Qwen2.5-7B where the model writes a note of the final facts before seeing the question. For Qwen2.5-32B, answering from the full history was correct in more cases than answering from the corrected facts, though the two use different prompts. Table 10 reports the gains, where a gain is the mean of the two trained seeds minus the baseline, paired within a case, and Table 11 lists the remaining errors.

History-dependent questions. With the full-history prompt of the repair evaluation, we count 32 history-dependent questions for Qwen2.5-7B, and 15 of them are also among the 94 of §3.2. The trained interpreter answers all 32 and all 94 correctly under every history, as it does the 9 from Qwen2.5-32B and the 41 from Gemma-2-27B, and all 517 check questions stay correct in each of the six runs. The 9 from Qwen2.5-32B are also answered correctly by the Gemma-2-27B interpreter.

Model	Seed	Refusals	Wrong operation
Qwen2.5-7B	0	3	1
	1	0	1
Qwen2.5-32B	0	0	0
	1	2	0
Gemma-2-27B	0	0	1
	1	0	1

Table 11: **Errors that remain with the trained interpreter**, out of 901 questions.

G History isolation and interpreter choice: details

Everything in this appendix is for Qwen2.5-7B on the 128 cases and four histories of §5.3, and it gives the details of §5.4.

Showing the interpreter the history. For this test, the dictionary, the question, the candidate operations and the code are all kept the same, and we use both trained interpreters without retraining them. The only change is that the original history, with its corrections, is appended after everything the interpreter normally reads. A line above it says that this text is not authoritative and that only the current entries are to be used as facts. As a comparison, we append text with the same lines in the same place, but with every statement given its final value (Table 12). With the history, the share of cases with every question correct falls by 41.8 percentage points on average over the two seeds, with a 95% interval of 36.3 to 47.3. This fall is 34.0 points larger than with the final facts, with an interval of 28.9 to 39.1. The two seeds differ, falling by 68.8 points, with an interval of 60.9 to 76.6, and by 14.8 points, with an interval of 9.4 to 21.1. Every new error is a refusal and not an outdated answer. The two added texts have the same number of tokens in 5,798 of 12,198 pairs.

Trained comparisons. We trained the two other models as adapters from the base model with the settings of Appendix F, using the same 12,288 training examples in the same order and two seeds each. Both read the current entries and the question in the form the interpreter reads them, but without a candidate operation. The first is trained to write the answer. The second is instead trained to write an operation, which is the entity it starts at and the relations it follows. The same code then runs what the model wrote (Table 13). On average over its seeds, we find the interpreter to be 1.17 percent-

Added text	Seed	Cases	Multi-hop	Checks	Changes
None	0	124	380	517	0
	1	127	383	517	0
Original history	0	36	366	392	122
	1	108	379	496	23
Final facts only	0	107	378	497	6
	1	124	382	515	1

Table 12: **Showing the trained interpreter more than the dictionary.** Cases are out of 128, multi-hop questions out of 384 and check questions out of 517, where each counts only if it is correct under all four histories. Changes is the number of the 901 questions answered under some histories and refused under others.

Answered by	Seed	Cases	Multi-hop	Checks	Before updates
Interpreter	0	124	380	517	1,522
	1	127	383	517	1,524
Writes operation	0	126	383	516	1,532
	1	126	384	515	1,530
Writes answer	0	126	383	516	1,266
	1	122	382	513	1,068
MeLLo, adapted	–	45	287	466	1,166
GMeLLo, six facts	–	91	323	517	1,364
GMeLLo, all facts	–	89	321	516	1,378
GMeLLo, text edits	–	46	236	474	1,000

Table 13: **Trained comparisons for Qwen2.5-7B.** The first three columns are as in Table 12. The last column is the number of correct answers to the multi-hop questions, out of 1,536, when each history's dictionary from before the updates is given instead. The three GMeLLo rows are our untrained versions of the method.

age points above the model that writes the answer, with a 98.75% interval of −1.95 to 4.69. It is 0.39 points below the model that writes the operation, with an interval of −3.12 to 2.34. In both seeds the interpreter keeps all 517 check questions correct, while every error the other two make on check questions is about the rest of the chain. Of the 94 history-dependent questions of §3.2, the interpreter repairs all 94, while each of the other two repairs 93. Since they read the same dictionary, all of the models in Table 13 give the same output under all four histories for all 901 questions.

MeLLo and GMeLLo. MeLLo follows the loop of its official code. The model writes a subquestion with a tentative answer, one fact is retrieved, and the model then checks its answer against that fact. We used Qwen2.5-7B as the model and the final facts of the case as the memory being retrieved from, while also showing the model all of those facts. Its prompt has four demonstrations that we built from training cases, and we allowed four rounds with one further call for the final answer. MeLLo differs from the interpreter by 62.89 percentage points, with a 98.75% interval of 52.34 to 73.05. For GMeLLo (Chen et al., 2024), we could not find the authors' code, so we followed the method as specified in its paper. Qwen2.5-7B is given the question, a fixed list of 37 relations and four demonstrations from training cases, and writes a chain of relations for the question. Code then turns this chain into a query over a graph holding the facts of the case, where the published method queries Wikidata instead. If the query returns an answer, that answer is used, and otherwise the method falls back to answering from retrieved facts. We tested three versions (Table 13). The first has code apply the updates to the graph, with a fallback that retrieves six of the final facts. The second is the same except that its fallback receives all of the final facts. The third has the model itself read the updates from the correction sentences and write them to the graph. We find that the first two versions are correct in 91 and 89 of the 128 cases, or 71% and 70%, compared to 124 and 127 for the two seeds of the interpreter. This puts the interpreter 26.95 percentage points above the first version, with a 97.5% interval of 18.75 to 35.55, and 28.52 above the second, with an interval of 19.92 to 37.50. The third version is correct in 46 cases. For the first two versions, the query returns an answer for 5,158 of the 6,942 questions each version is asked, and 5,148 of these answers are correct. The interpreters are trained while these versions of GMeLLo are not, and we chose the GMeLLo prompt after development runs, so this compares whole systems on cases we had already examined.

The dictionary before the updates. We also give each model the copy of the dictionary from before the updates under each history, and ask the same 384 multi-hop questions, for 1,536 answers per seed. In 768 of them the correct answer is not the one from after the updates. The model that writes the answer gives the answer from after the updates in 226 of those 768 for one seed and 440 for the other, which accounts for most of its errors. Neither of the models that have code run an operation ever does. Both of them answer at least 99% of the 1,536 questions correctly from the dictionary they are given, while the model that writes the answer falls to 76%.